

توثيق مشروع بيت المندي

الوثيقة التقنية والمعمارية الشاملة

تطوير وإنتاج: Esnaad Tech

يونيو 2026

الملخص التنفيذي – مشروع بيت المندي

نظرة عامة على المشروع

اسم المشروع: بيت المندي – منصة الطلبات الرقمية

الشركة المطورة: Esnaad Tech

تاريخ الإصدار: يونيو 2026

الإصدار: 1.0.0

رابط الإنتاج: <https://baitalmandi.vercel.app>

المستودع: <https://github.com/engmohammedalshaibani2/baitalmandi>

وصف المشروع

منصة بيت المندي هي تطبيق ويب متكامل من الجيل التالي، تم تصميمه وتطويره بالكامل لخدمة مطعم بيت المندي في صنعاء، اليمن. يوفر المشروع تجربة طلب رقمية متكاملة تشمل:

- واجهة عملاء تفاعلية لاستعراض القائمة وتقديم الطلبات
- نظام توصيل ذكي مع حساب المسافة والأسعار آلياً
- لوحة إدارة متكاملة لإدارة كافة عمليات المطعم
- نظام تقارير وتحليلات متقدم
- نظام أمان متعدد الطبقات

الأهداف الاستراتيجية

الهدف	التفاصيل
رقمنة عمليات المطعم	تحويل كافة عمليات الطلب من الهاتف/واتساب إلى منصة رقمية
تحسين تجربة العملاء	تمكين العملاء من الطلب والتتبع بسهولة تامة
تعزيز الكفاءة التشغيلية	تقليل الأخطاء البشرية في معالجة الطلبات
إتاحة البيانات والتحليلات	توفير لوحة تقارير لاتخاذ قرارات مبنية على البيانات
بناء قاعدة عملاء	بناء علاقة مستدامة مع العملاء عبر تتبع سجل طلباتهم

المزايا الرئيسية للنظام

للعلماء

- استعراض القائمة الكاملة بأوصاف عربية وإنجليزية
- طلب أصناف متعددة مع اختيار الحجم والكمية
- تطبيق العروض والحزم المخفضة بشكل تلقائي
- تحديد موقع التوصيل على الخريطة
- معرفة رسوم التوصيل قبل تأكيد الطلب
- تتبع حالة الطلب في الوقت الفعلي
- الطلب عبر الموقع أو واتساب

للمدارة

- لوحة تحكم مركزية بصلاحيات متدرجة
- إدارة كاملة للقائمة والأسعار والعروض
- إدارة الطلبات مع تتبع الحالات
- نظام تقارير متكامل (مبيعات، منتجات، عملاء، توصيل)
- تصدير التقارير بصيغ PDF و Excel
- إدارة معرض الصور والتقييمات
- إعدادات التوصيل والنظام

القيمة التشغيلية

- توفير الوقت: أتمتة حساب الأسعار والتوصيل يوفر دقائق لكل طلب
- تقليل الأخطاء: المعالجة الآلية تمنع أخطاء الطلبات اليدوية
- الشفافية: العملاء يرون تكلفة توصيلهم قبل الطلب
- البيانات: سجل كامل لكل طلب ومنتج ومسار
- التوسع: البنية المعمارية تدعم إضافة فروع متعددة

المقاييس التقنية

المقياس	القيمة
إطار العمل	Next.js 14 App Router
لغة البرمجة	TypeScript
قاعدة البيانات	PostgreSQL (Supabase)
ORM	Drizzle ORM
عدد الجداول	19 جدول
عدد صفحات المستخدم	7 صفحات
عدد صفحات الإدارة	11 صفحة
عدد API Routes	13+ مسار
عدد Server Actions	5 ملفات
بيئة النشر	Vercel

فريق المشروع

الدور	الشركة
التصميم والتطوير	Esnaad Tech
البنية التحتية	Supabase (قاعدة البيانات) + Vercel (النشر)

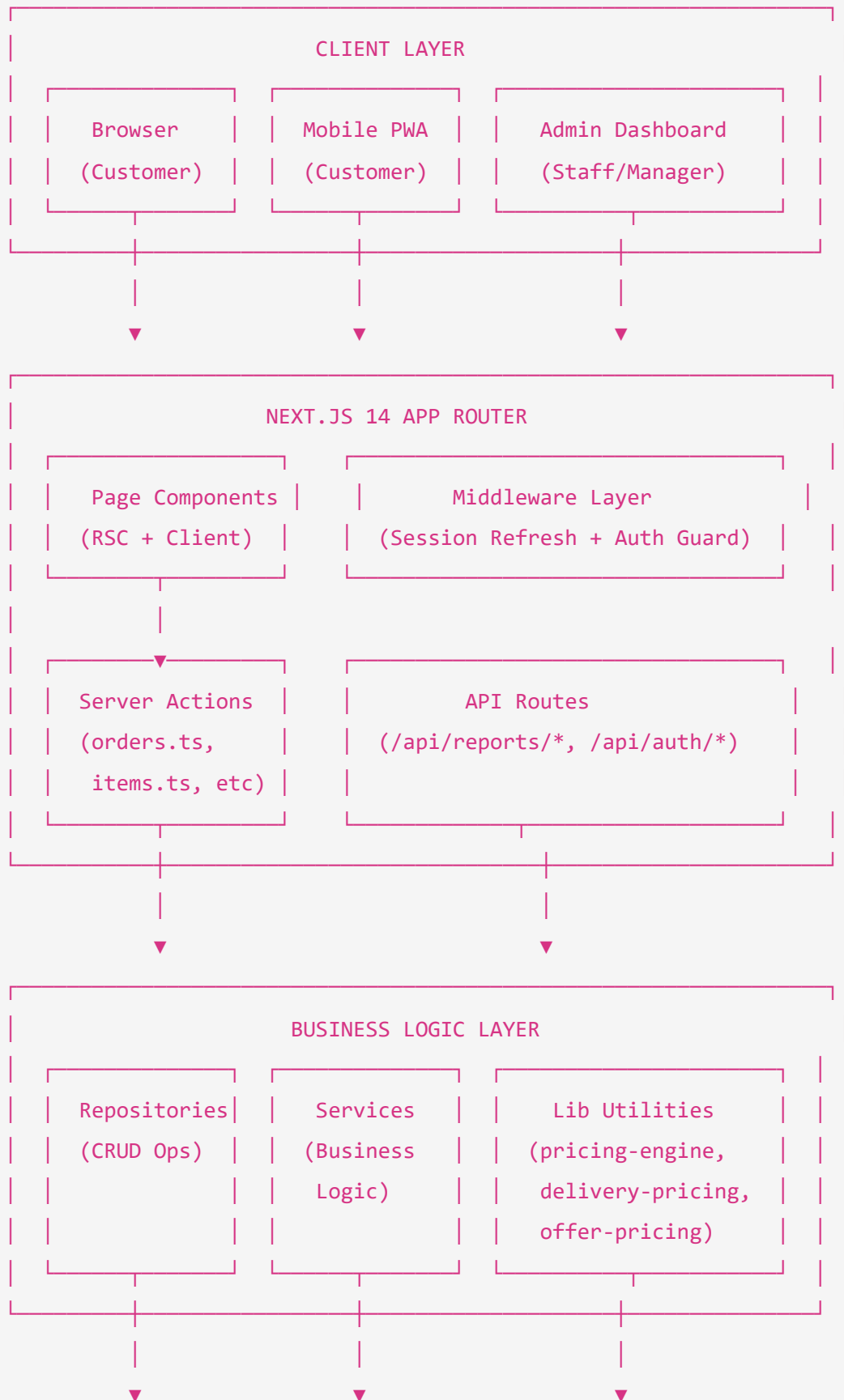
توثيق البنية المعمارية – مشروع بيت المندي

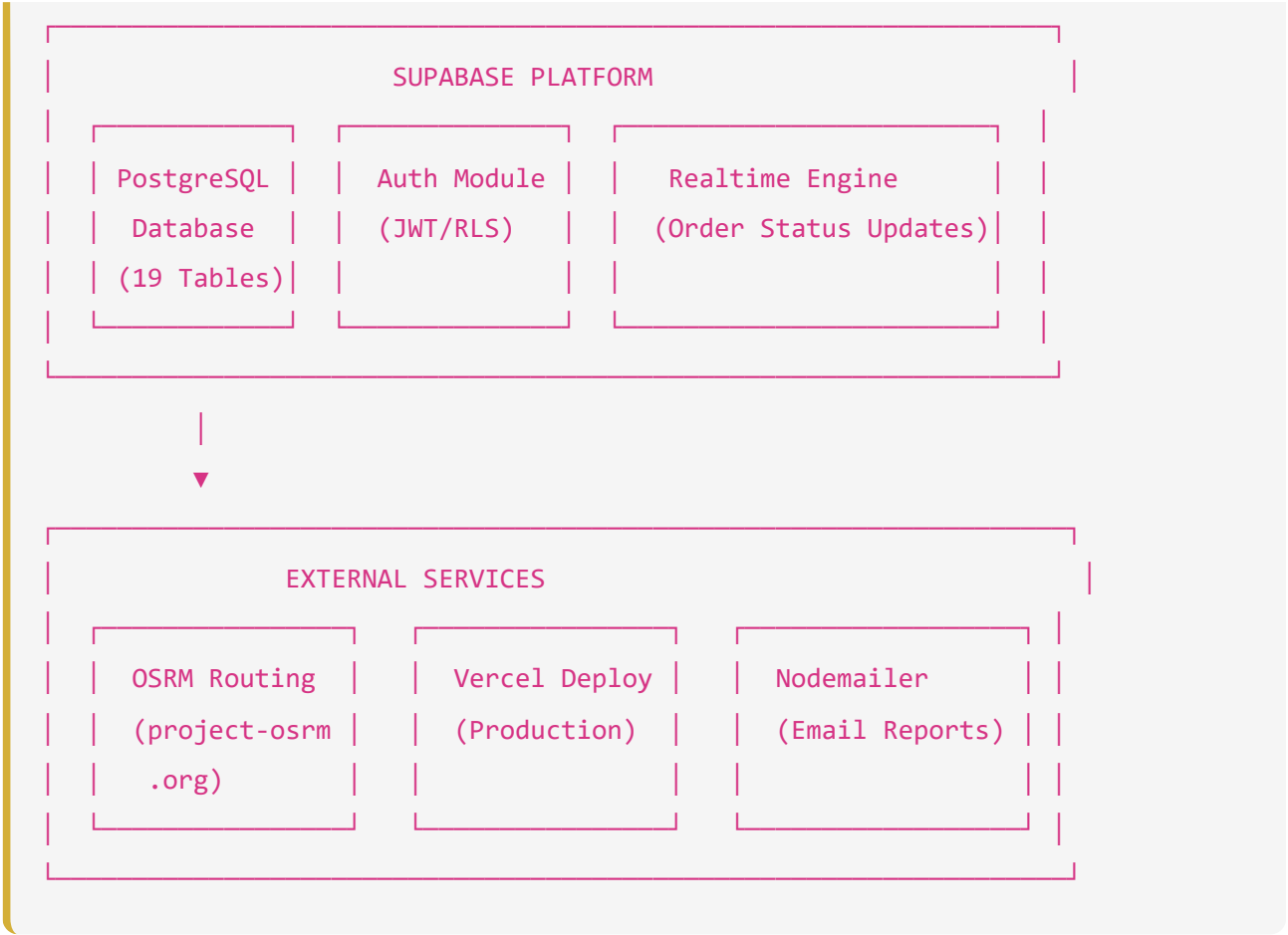
1. نظرة عامة على المعمارية

يعتمد المشروع على معمارية **Full-Stack Serverless** باستخدام Next.js 14 مع App Router. مما يجمع بين:

- **Server-Side Rendering (SSR)** لتحسين الأداء وتحسين محركات البحث
 - **Server Actions** لمعالجة العمليات من جانب الخادم بأمان
 - **Supabase** كـ Backend-as-a-Service يوفر قاعدة بيانات وتوثيقاً ومزامنة لحظية
-

2. مخطط المعمارية العالي المستوى (High-Level Architecture)





3. هيكلية المشروع الكاملة

```

baitalmandiwibapp/
├─ src/
│   ├─ app/                                # Next.js App Router
│   │   ├─ layout.tsx                      # Root layout (RTL، SEO، Providers)
│   │   ├─ page.tsx                       # (SSR) الصفحة الرئيسية
│   │   ├─ HomeClient.tsx                 # مكون الصفحة الرئيسية التفاعلي
│   │   ├─ globals.css                   # (27KB) نظام التصميم الكامل
│   │   ├─ error.tsx                     # معالجة الأخطاء
│   │   ├─ loading.tsx                   # شاشة التحميل
│   │   ├─ robots.ts                    # SEO: ملف robots
│   │   ├─ sitemap.ts                    # SEO: خريطة الموقع
│   │   ├─ menu/                         # صفحة القائمة
│   │   ├─ cart/                         # صفحة السلة والطلب
│   │   ├─ gallery/                      # معرض الصور
│   │   ├─ contact/                     # صفحة التواصل
│   │   ├─ my-orders/                   # طلباتي (بحث برقم الهاتف)
│   │   ├─ track-order/[orderId]/        # تتبع طلب محدد
│   │   ├─ t/[token]/                   # تتبع برمز التتبع
│   │   ├─ admin/                       # لوحة الإدارة
│   │   │   ├─ layout.tsx                # Admin Layout (SSR auth check)
│   │   │   ├─ AdminLayoutClient.tsx    # التفاعلية Sidebar واجهة
│   │   │   ├─ page.tsx                 # داشبورد الرئيسية
│   │   │   ├─ login/                   # صفحة تسجيل الدخول
│   │   │   ├─ orders/                  # إدارة الطلبات
│   │   │   ├─ menu/                    # إدارة الأطباق
│   │   │   ├─ categories/              # إدارة التصنيفات
│   │   │   ├─ offers/                  # إدارة العروض
│   │   │   ├─ gallery/                 # إدارة الصور
│   │   │   ├─ reviews/                 # إدارة التقييمات
│   │   │   ├─ delivery/                # إعدادات التوصيل
│   │   │   ├─ reports/                 # التقارير والتحليلات
│   │   │   └─ settings/                # إعدادات النظام
│   │   └─ api/                         # API Routes
│   │       ├─ auth/                    # (logout) مصادقة
│   │       └─ reports/                  # تقارير متعددة

```

```

├── dashboard/
├── sales/
├── orders/
├── products/
├── customers/
├── offers/
├── delivery-analytics/
├── compare/
├── audit/
├── invoices/
├── schedule/
├── cron/
└── refresh-mv/

├── actions/ # Server Actions
│   ├── orders.ts # إنشاء وإدارة الطلبات
│   ├── items.ts # إدارة الأصناف
│   ├── categories.ts # إدارة التصنيفات
│   ├── orders-offers.ts # عمليات الطلبات والعروض
│   └── settings.ts # إعدادات النظام
│
├── db/ # Database Layer
│   ├── schema.ts # Drizzle ORM Schema (19 Tables)
│   ├── index.ts # اتصال قاعدة البيانات
│   └── rls.sql # Row Level Security سياسات
│
├── repositories/ # Data Access Layer
│   ├── adminRepository.ts # عمليات مستخدمي الإدارة
│   ├── categoryRepository.ts # عمليات التصنيفات
│   ├── galleryRepository.ts # عمليات الصور
│   ├── menuRepository.ts # عمليات القائمة
│   ├── offerRepository.ts # عمليات العروض
│   ├── orderRepository.ts # عمليات الطلبات
│   ├── reviewRepository.ts # عمليات التقييمات
│   └── settingsRepository.ts # عمليات الإعدادات
│
├── lib/ # Utility Libraries
│   └── pricing-engine.ts # محرك الأسعار المركزي

```

	delivery-pricing.ts	# حساب رسوم التوصيل
	delivery-routing.ts	# توجيه OSRM
	offer-pricing.ts	# حساب أسعار العروض
	location-validation.ts	# التحقق من الموقع (Haversine)
	permissions.ts	# نظام الصلاحيات
	validation.ts	# التحقق من المدخلات
	logger.ts	# نظام السجلات
	whatsapp-message.ts	# بناء رسائل واتساب
	exportExcel.ts	# تصدير Excel (16KB)
	printReport.ts	# طباعة التقارير (26KB)
	formatUtils.ts	# أدوات التنسيق
	settings-context.tsx	# سياق الإعدادات
	components/	# مكونات React
	layout/	# Navbar, Footer, LoadingScreen
	ui/	# Toast, مكونات UI
	admin/reports/	# مكونات تقارير الإدارة
	invoice/	# مكونات الفواتير
	context/	# React Contexts
	store/	# Zustand State Management
	realtime/	# Supabase Realtime
	schemas/	# Zod Validation Schemas
	services/	# Business Services
	utils/	# Supabase Client Utils
	validators/	# Input Validators
	middleware.ts	# Auth Middleware
	public/	# Static Assets
	logo.jpg	
	Esnaad Tech Logo.svg	
	supabase/	# Supabase Config
	drizzle.config.ts	# Drizzle Configuration
	next.config.js	# Next.js Configuration
	tailwind.config.js	# Tailwind Configuration
	package.json	# Dependencies

4. طبقات المعمارية (Layered Architecture)

الطبقة 1: طبقة العرض (Presentation Layer)

- **React Server Components (RSC)**: تُنفَّذ على الخادم، مُحسَّنة للأداء
- **Client Components**: تعمل في المتصفح، تتعامل مع التفاعل
- **App Router**: نظام التوجيه المبني على ملفات 14 Next.js

الطبقة 2: طبقة المنطق التجاري (Business Logic Layer)

- **Server Actions**: دوال خادم تُستدعى مباشرة من المكونات
- **API Routes**: نقاط نهاية HTTP للتقارير والمصادقة
- **Lib Utilities**: مكتبات الحساب (pricing, routing, validation)

الطبقة 3: طبقة الوصول للبيانات (Data Access Layer)

- **Repositories**: تجريد عمليات قاعدة البيانات
- **Drizzle ORM**: تحويل TypeScript إلى SQL بأمان
- **Supabase Client**: اتصال PostgreSQL المُدار

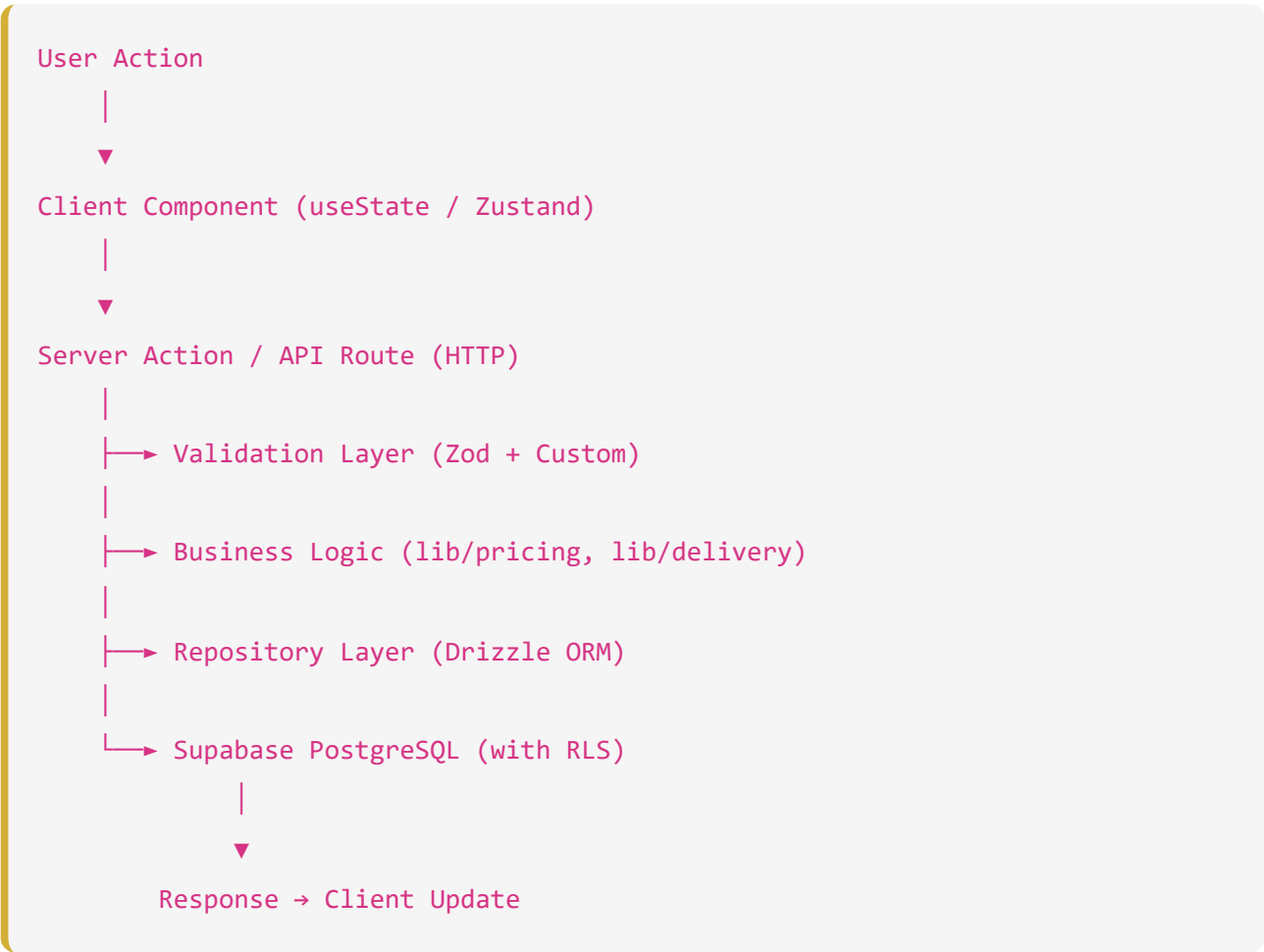
الطبقة 4: طبقة قاعدة البيانات (Database Layer)

- **PostgreSQL**: قاعدة البيانات الرئيسية
- **RLS Policies**: أمان على مستوى الصف
- **Supabase Auth**: إدارة المصادقة والجلسات

5. نمط إدارة الحالة (State Management)

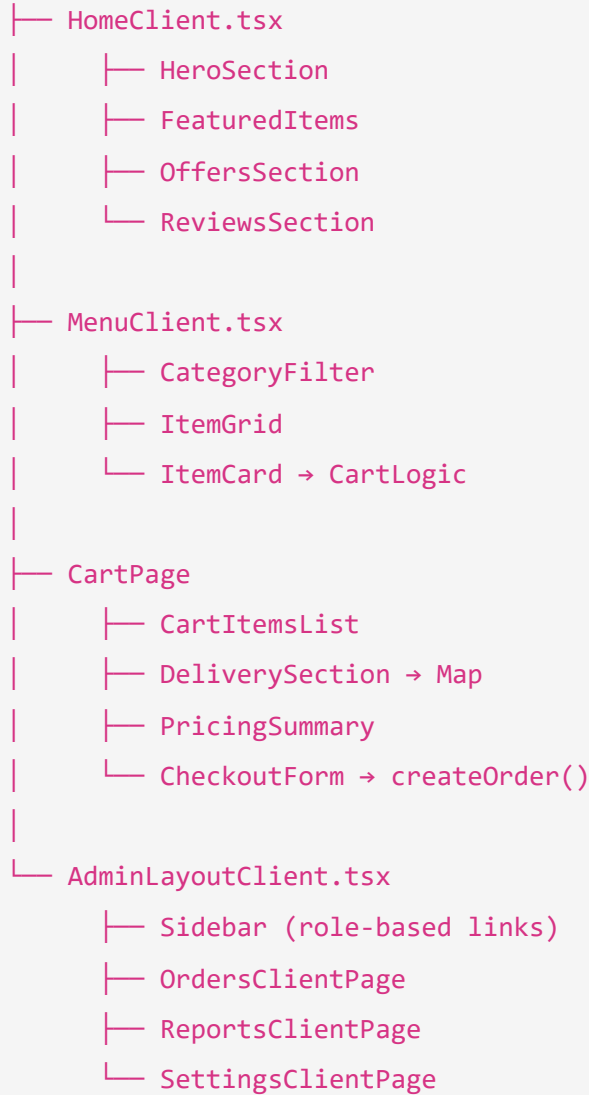
النمط	الأداة	الاستخدام
Server State	React Server Components	بيانات قاعدة البيانات
Global Client State	Zustand	سلة المشتريات، حالة المستخدم
UI State	React useState	حالات المكونات المحلية
Settings State	React Context	إعدادات الموقع العامة
Realtime State	Supabase Realtime	تحديثات حالة الطلبات

6. تدفق البيانات



7. مخطط المكونات (Component Diagram)

Pages



8. التقنيات المستخدمة

التقنية	الإصدار	الغرض
Next.js	14.2.3	إطار العمل الأساسي
React	18.3.1	بناء واجهة المستخدم
TypeScript	5.4.5	الكتابة الصارمة
Supabase JS	2.106.1	Client SDK
Supabase SSR	0.3.0	Server-Side Auth
Drizzle ORM	0.30.10	ORM لقاعدة البيانات
Tailwind CSS	3.4.3	نظام التنسيق
Framer Motion	11.2.6	الرسوم المتحركة
Leaflet	1.9.4	خريطة التوصيل
Recharts	3.8.1	رسوم التقارير
Zustand	4.5.2	إدارة الحالة
Zod	4.4.3	التحقق من البيانات
ExcelJS	4.4.0	تصدير Excel
jsPDF	4.2.1	تصدير PDF
Nodemailer	8.0.10	إرسال التقارير
Turf.js	7.3.5	حسابات جغرافية
OSRM	External	حساب مسارات التوصيل

توثيق قاعدة البيانات – مشروع بيت المندي

1. نظرة عامة على قاعدة البيانات

يعتمد النظام على **PostgreSQL** المستضافة في **Supabase**، وتدار بالكامل عبر **Drizzle ORM**. تتكون القاعدة من 19 جدولاً مرتبطاً بعلاقات قوية ومحمية بـ Row Level Security (RLS).

2. المخطط الكياني العلائقي (ERD Diagram)

```
erDiagram
    users ||--o{ orders : "places"
    admin_users ||--o{ audit_logs : "creates"
    admin_users ||--o{ order_status_history : "updates"

    categories ||--o{ items : "contains"
    items ||--o{ item_prices : "has"
    items ||--o{ offer_items : "included_in"

    offers ||--o{ offer_items : "contains"
    offers ||--o{ order_offers : "applied_to"

    cart_sessions ||--o{ cart_items : "contains"
    cart_items }o--|| items : "references"
    cart_items }o--|| item_prices : "references"

    orders ||--o{ order_items : "contains"
    orders ||--o{ order_offers : "has_offers"
    orders ||--o{ order_status_history : "has_history"

    order_offers ||--o{ order_offer_items : "contains_items"
```


3. تفاصيل الجداول

3.1. جداول المستخدمين والصلاحيات

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
users	بيانات العملاء الأساسية	orders	تسجيل الطلبات وحفظ العناوين
admin_users	حسابات الإدارة وصلاحياتهم	audit_logs, order_status_history	لوحة الإدارة وتسجيل الحركات

3.2. جداول القائمة والمنتجات

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
categories	تصنيفات الأطباق	items	صفحات القائمة والإدارة
items	الأطباق والمنتجات	categories, item_prices	القائمة، السلة، العروض
item_prices	أسعار الأصناف حسب الحجم	items, cart_items	محرك الأسعار

3.3. جداول العروض

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
offers	العروض والتخفيضات النشطة	offer_items	صفحة العروض والسلة
offer_items	الأصناف المكونة للعرض	offers, items, item_prices	محرك أسعار العروض

3.4. جداول السلة (المؤقتة)

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
<code>cart_sessions</code>	جلسات التسوق المؤقتة	<code>cart_items</code>	عربة التسوق
<code>cart_items</code>	محتويات عربة التسوق	<code>cart_sessions, items</code>	عربة التسوق

3.5. جداول الطلبات الرئيسية

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
<code>order_sequences</code>	ترقيم الطلبات (-BAM- (DATE-XXXX	مستقل	عملية إنشاء الطلب
<code>orders</code>	البيانات الرئيسية للطلب	<code>users, order_items, order_offers</code>	لوحة الإدارة والمستخدم
<code>order_items</code>	الأصناف المطلوبة (سعر ثابت)	<code>orders</code>	تفاصيل الطلب والفواتير
<code>order_offers</code>	العروض المطبقة على الطلب	<code>orders, offers</code>	تفاصيل الطلب والفواتير
<code>order_offer_items</code>	أصناف العرض المباعة	<code>order_offers</code>	الفواتير والمخزون
<code>order_status_history</code>	سجل تغير حالات الطلب	<code>orders, admin_users</code>	تتبع الطلبات والإدارة

3.6. جداول التتبع وإدارة النظام

اسم الجدول	الغرض	العلاقات	مستخدم بواسطة
customer_tokens	أرقام تتبع الطلبات المستقلة	مستقل	نظام التتبع بدون تسجيل
gallery_images	صور معرض المطعم	مستقل	صفحة المعرض والإدارة
reviews	تقييمات العملاء	مستقل	الصفحة الرئيسية والإدارة
site_settings	إعدادات النظام (JSON)	مستقل	التوصيل والأسعار والموقع
branches	فروع المطعم	مستقل	صفحة التواصل
audit_logs	سجل حركات الإدارة	admin_users	التقارير الأمان
scheduled_reports	إعدادات التقارير المجدولة	مستقل	Cron Jobs

4. المفاتيح والقيود (Constraints & Indexes)

1. الترتيم التسلسلي (Order Sequences):

- يستخدم قيد `sequence_date` ك `UNIQUE` لضمان عدم تكرار الترتيم في نفس اليوم.
- يعتمد على عملية `UPSERT` ذرية (Atomic) لضمان التزامن.

2. الأسعار (Prices):

- نوع البيانات `numeric` يُستخدم في كل المبالغ المالية لمنع أخطاء التقريب الناتجة عن أنواع الفاصلة العائمة (`float`).

3. الحذف الآمن (Soft Deletes):

- الجداول الحساسة (مثل `orders`) تحتوي على أعمدة `is_deleted` و `deleted_at` ولا يتم حذف بياناتها فعلياً، بل تُخفى في واجهات القراءة.

4. المعرفات (UUIDs):

- جميع الجداول (باستثناء `customer_tokens`) تستخدم `UUIDv4` العشوائي لتوفير أمان إضافي ضد تخمين المعرفات.

5. النسخ الاحتياطية للسجلات (Snapshots)

يعتمد تصميم قاعدة البيانات على نمط "Snapshotting" في الطلبات:

- يتم أخذ نسخة من **الأسعار والأسماء** للأصناف وحفظها في **order_items** و **order_offers**.
- هذا يضمن أن تعديل اسم أو سعر منتج في المستقبل **لا** يغير الفواتير التاريخية القديمة.

توثيق الأمان والصلاحيات – مشروع بيت المندي

1. نموذج الأمان المعماري (Security Model)

يتبع مشروع بيت المندي مبدأ "الدفاع في العمق" (Defense in Depth) باستخدام عدة مستويات أمنية:

1. **Middlewares (Edge Security)**: لحماية المسارات على مستوى الخادم.
2. **Server Actions (Application Security)**: عمليات المصادقة المضمنة داخل الكود.
3. **RLS Policies (Database Security)**: أمان على مستوى صفوف قاعدة البيانات (PostgreSQL).
4. **Permissions System (Role-Based Access)**: الصلاحيات المتدرجة (RBAC).

2. المصادقة والجلسات (Authentication & Sessions)

- المزود: يعتمد على Supabase Auth.
- الجلسات: يتم إرسال JWT في الـ Cookies الآمنة (HttpOnly, Secure).
- Middleware: ملف `middleware.ts` في جذر المشروع يقوم بتحديث الجلسة وفحص المسارات المحمية (`admin, /api/admin, /api/reports/`).

3. الصلاحيات المتدرجة (Role-Based Access Control)

يتم تقسيم المديرين إلى عدة أدوار داخل النظام (مُعرّفة في Enum `admin_role`):

الدور (Role)	الوصف	الصلاحيات
developer	المطور الرئيسي	وصول كامل 100% وإمكانية إضافة مديرين وتعديل إعدادات النظام وتعديل الأكواد
manager	مدير المطعم	وصول كامل لجميع العمليات (عدا إدارة المطورين)
admin	المشرف العام	إدارة الأقسام والطلبات، بدون الوصول لإعدادات حساسة.
order_manager	مدير الطلبات	مخصص حصرياً لصفحة <code>admin/orders/</code> واستلام وتغيير حالات الطلب.

آلية التحقق: يتم فحص الصلاحيات عبر مكتبة `lib/permissions.ts` في جانب العميل والخادم، بالإضافة إلى دوال قاعدة البيانات المساعدة (`is_admin()`, `get_admin_role()`).

4. أمان قاعدة البيانات (Row Level Security - RLS)

جميع الجداول ممكّن عليها (`ENABLE ROW LEVEL SECURITY`) RLS.

سياسات القراءة (SELECT):

- **الجمهور (Public):** يمكنهم قراءة التصنيفات (`categories`)، الأطباق (`items`)، الأسعار (`item_prices`)، العروض (`offers`)، والمراجعات المعروضة.
- **العملاء:** يمكنهم قراءة طلباتهم فقط عبر `auth.uid() = customer_id`.
- **الإدارة:** لديهم صلاحية قراءة كل شيء باستخدام دالة `is_admin()`.

سياسات الكتابة (INSERT/UPDATE/DELETE):

- **الجمهور:** يمكنهم تسجيل التقييمات وإنشاء الجلسات المؤقتة، وإنشاء طلب إذا كان `customer_id` مساوياً لحسابهم أو `NULL` (للزوار).
- **الإدارة:** العمليات التشغيلية (المنتجات، العروض، الإعدادات) محصورة بالأدوار `developer` و `manager`.
- **مدير الطلبات:** صلاحيته محصورة في تحديث حالة الطلبات (`UPDATE` على `orders` و `INSERT` في `order_status_history`).

القيود الخاصة (Hard Deletes Prevention):

هناك قاعدة (Rule) لمنع الحذف الفعلي للطلبات لضمان النزاهة المالية: **CREATE RULE**
;prevent_orders_delete AS ON DELETE TO orders DO INSTEAD NOTHING

5. سجلات التدقيق (Audit Logs)

تُسجّل أي حركة مفصلية داخل النظام (تعديل الطلب، إلغاء الطلب، تغيير الحالات، وتغيير الإعدادات) في جدول **audit_logs**.

- يحتوي الجدول على: المشرف الذي قام بالعملية، نوع العملية، تفاصيل التغيير، والكيان المعدل.
 - هذا السجل يتيح المساءلة التامة (Accountability).
-

6. تأمين Server Actions

كل دوال الخادم الموجودة في مجلد **src/actions** تقوم بـ:

1. إنشاء **createClient()** لفحص جلسة المستدعي.
 2. استخدام محرك الأسعار المركزي الموجود في **lib/pricing-engine.ts** وحساب الأسعار على الخادم حصرياً (Sever-Side Calculation)، مما يمنع التلاعب بالأسعار من طرف العميل (Client-Side Tampering).
 3. تغليف عمليات الإنشاء المعقدة بـ **Drizzle transaction()** لضمان "إما نجاح العملية بالكامل أو إلغاؤها".
-

7. مخطط تدفق مصادقة الإدارة (Admin Authorization Flow)

```
sequenceDiagram
    participant User as Admin User
    participant App as Next.js App
    participant Middleware as Auth Middleware
    participant Action as Server Action
    participant Supabase as Supabase (RLS)

    User->>App: Login with Email/Password
    App->>Supabase: Auth Request
    Supabase-->>App: JWT Session (Cookies)

    User->>App: Request /admin/orders
    App->>Middleware: Intercept Request
    Middleware->>Supabase: Validate Session
    Supabase-->>Middleware: Session Valid
    Middleware-->>App: Allow Request

    App->>Action: updateOrderStatus(id)
    Action->>Action: Validate Role Constraint
    Action->>Supabase: Update Query (with Auth Context)
    Supabase->>Supabase: Check RLS Policies
    Supabase-->>Action: Success / Failure
    Action-->>App: Return Result
```


توثيق دورة حياة الطلبات – مشروع بيت المندي

1. نظرة عامة على نظام الطلبات

يعد نظام الطلبات في مشروع بيت المندي هو القلب النابض للتطبيق. يعتمد النظام على معمارية ذرية (Atomic Transactions) لمعالجة البيانات، ويعتمد على الخادم حصرياً (Server-Side) لحساب التكاليف لضمان منع الاحتيال أو التلاعب بالأسعار.

2. دورة حياة الطلب (Order Lifecycle)

```
stateDiagram-v2
    [*] --> Draft : Customer Adding to Cart
    Draft --> Validation : Checkout Submission

    state Validation {
        [*] --> CheckPrices
        CheckPrices --> CheckDelivery
        CheckDelivery --> CheckOffers
        CheckOffers --> AtomicTransaction
    }

    Validation --> pending : Success
    Validation --> Draft : Failure (Error Displayed)

    pending --> confirmed : Admin Confirms
    pending --> cancelled : Admin/Auto Cancel

    confirmed --> preparing : Kitchen Starts
    preparing --> on_the_way : Driver Picked Up
    on_the_way --> delivered : Order Completed

    delivered --> [*]
    cancelled --> [*]
```

3. محرك الأسعار المركزي (Pricing Engine)

النظام لا يعتمد مطلقاً على حسابات الأسعار المرسلّة من جهة العميل (المتصفح). يتم إرسال "المعرفات" والكميات فقط.

ملف: `src/lib/pricing-engine.ts`

- يقبل المدخلات ويعالج الحالات غير المتوقعة (`NaN`, `null`, `undefined`) عبر دالة `safeNumber`.
- يعيد حساب الإجمالي (Subtotal) بناءً على سعر المنتج في قاعدة البيانات.
- يدمج رسوم التوصيل، الضرائب، والخصومات لإنتاج `totalAmount`.

- يتم استخدام الدالة `calculateOrderTotals` في الواجهة الخلفية فقط (Server Actions).

4. ترقيم الطلبات (Order Number Generation)

يتم إنتاج رقم تسلسلي للطلب بصيغة `BAM-YYYYMMDD-XXXX` ليكون سهل القراءة للعملاء وممثلاً لتاريخ اليوم. العملية تعتمد على:

1. الإقفال المتبادل (Concurrency Control).
2. عملية `UPSERT` متكررة على جدول `order_sequences` لحجز الرقم التالي (Sequence Allocation).
3. آلية تكرار (Retry Mechanism) تصل لـ 10 محاولات في حالة الصدام، مع خطة بديلة (UUID Fallback) في حالة فشل التسلسل.

5. الحفظ باستخدام Transactions (Drizzle)

عملية حفظ الطلب داخل `src/actions/orders.ts` مجمعة في Transaction واحد، تشمل:

1. التحقق من الأسعار والحد الأدنى للطلب ورسوم التوصيل.
2. إنشاء توكن التتبع (Tracking Token) في `customer_tokens` (للزوار).
3. `Insert Orders`: إنشاء السجل الرئيسي في جدول `orders`.
4. `Insert Items`: تسجيل الأصناف بأسعارها وقت الشراء في (Snapshot) `order_items`.
5. `Insert History`: تسجيل الحالة الافتتاحية في `order_status_history`.
6. `Insert Offers`: (اختياري) تفكيك العروض وحزم المنتجات إلى عناصر وحفظها في `order_offers` و `order_offer_items`.

إذا فشلت أي خطوة من هذه الخطوات، يتم التراجع عن (Rollback) كافة الخطوات السابقة، مما يمنع مشكلة "البيانات اليتيمة" (Orphan Data).

6. التعامل مع العروض داخل الطلب (Offers Processing)

عند تقديم عرض (Bundle أو Discount)، يتم معالجته بواسطة `src/lib/offer-pricing.ts`.

- يتم حساب قيمة التخفيض بدقة (سواء كان مبلغاً ثابتاً، أو نسبة، أو عنصراً مجانياً).
- يتم تقسيم الإجمالي المخفض وتوزيعه ليحسب في إجمالي الفاتورة بطريقة آمنة.
- يتم أخذ "لقطة" (Snapshot) لمحتوى العرض في وقت الطلب حتى لا يتأثر لو تغير محتوى العرض لاحقاً.

7. التتبع بدون تسجيل دخول (Guest Tracking)

النظام يسمح للمستخدم بالطلب دون الحاجة لإنشاء حساب (Guest Checkout).

- يتم إنشاء رمز سري عشوائي (Tracking Token).
- يُربط الرمز برقم هاتف العميل عبر جدول `customer_tokens`.
- يستخدم العميل الرابط `track-order/[orderId]/` ورقم الهاتف لعرض حالة الطلب المباشرة.
- يستفيد النظام من **Supabase Realtime** لتحديث صفحة التتبع لدى العميل فور قيام الإدارة بتغيير الحالة دون الحاجة لإعادة تحميل الصفحة.

توثيق نظام التوصيل – مشروع بيت المندي

1. نظرة عامة

يملك مشروع بيت المندي نظام حساب توصيل معقد (Dynamic Delivery Pricing) يستند إلى الموقع الجغرافي للمطعم والعميل، ويستخدم نظام التوجيه (Routing) الفعلي للمركبات مع طبقات من الرسوم المتغيرة حسب ظروف الوقت والطقس.

2. منطق حساب المسافة (Distance Calculation)

يقوم النظام بحساب المسافة باستخدام نهجين متوازيين:

1. المسار الفعلي (OSRM Routing):

- يستخدم خدمة router.project-osrm.org المجانية.
- يعيد المسافة الفعلية بالسيارة (Road Distance) والوقت المقدر (Duration).
- يتم حماية الاستدعاء بوقت أقصى (Timeout) مقداره 8 ثوان.

2. الخط المستقيم (Haversine Formula):

- يُستخدم للتحقق (Validation) وكخطة بديلة (Fallback) إذا تعطل OSRM.
- يتم احتساب المسافة الجغرافية المستقيمة، وتُضرب في معامل طريق [road_factor](#) (افتراضياً 1.5) لتقدير مسافة الطريق الفعلية.
- سبب التحقق: منع أي تلاعب يرسل إحداثيات خاطئة تجعل OSRM يُنتج مساراً غير منطقي.

3. محرك حساب الرسوم (Delivery Pricing Engine)

يوجد محرك تسعير وحيد (Single Source of Truth) موجود في [src/lib/delivery-pricing.ts](#). لا يثق النظام أبداً برسوم التوصيل المرسلة من العميل بل يعيد حسابها في الخادم.

معادلة حساب الرسوم الأساسية:

- إذا كانت $Distance \leq IncludedDistance$ (المسافة المشمولة):
- الرسوم = $BaseFee$ (الرسوم الأساسية).

- إذا كانت $Distance > IncludedDistance$:
 ◦ الرسوم = $BaseFee + (Distance - IncludedDistance) \times ExtraFeePerKm$ (رسوم الكيلومتر الإضافي).

الحدود القصوى:

- النظام يرفض أي طلب تتجاوز مسافته الـ $MaxDeliveryDistance$ (حد نطاق التوصيل).

4. الرسوم الإضافية (Surcharges)

يشتمل المحرك على رسوم ديناميكية:

1. رسوم الطقس (Weather Fee):

- يمكن تفعيلها من لوحة الإعدادات.
- إذا فُعلت، يضاف مبلغ ثابت مقطوع لإجمالي رسوم التوصيل لتعويض المناديب أثناء سوء الأحوال الجوية.

2. رسوم أوقات الذروة (Peak Hours Fee):

- يتم تحديد أوقات بداية ونهاية الذروة وأيام محددة (مثل الخميس والجمعة).
- تحسب كنسبة مئوية (Percentage) من **الرسوم الأساسية** فقط (وليس الإجمالي).
- النظام يتحقق من وقت الطلب الحالي في خادم الإنتاج مقارنة بنطاق الذروة.

5. حدود التوصيل في صنعاء (Location Validation)

بالإضافة للمسافة، يحتوي ملف `src/lib/location-validation.ts` على تقريب لمضلع الحدود الجغرافية لمدينة صنعاء باستخدام خوارزمية (Ray-Casting Algorithm).

- يُستخدم هذا لضمان أن النطاق يقع فعلياً داخل العاصمة ولا يقبل إحداثيات عشوائية أو بعيدة غير مخدمة.

6. مخطط تدفق احتساب التوصيل (Delivery Calculation Flow)

```
sequenceDiagram
```

```
    participant User as Customer (Browser)
```

```
    participant Action as Server Action (orders.ts)
```

```
    participant Validation as Location Validation
```

```
    participant Routing as Routing (OSRM)
```

```
    participant Pricing as Pricing Engine
```

```
    User->>Action: Submit Checkout (Lat, Lng)
```

```
    Action->>Validation: Validate within Max Radius
```

```
    Validation-->>Action: OK
```

```
    Action->>Routing: Fetch Route (Rest.Lat/Lng -> User.Lat/Lng)
```

```
    Routing-->>Action: Returns Road Distance & Duration
```

```
    alt OSRM Fails or Result Irregular
```

```
        Routing-->>Action: Use Haversine × Road Factor
```

```
    end
```

```
    Action->>Pricing: calculateDeliveryFee(Distance, Settings, Time)
```

```
    Pricing->>Pricing: Calculate Base Fee
```

```
    Pricing->>Pricing: Apply Weather Fee (if active)
```

```
    Pricing->>Pricing: Apply Peak Multiplier (if active & time match)
```

```
    Pricing-->>Action: Return Total Delivery Fee
```

```
    Action->>Action: Finalize Order Total & Insert Database
```

توثيق لوحة الإدارة – مشروع بيت المندي

1. نظرة عامة

تعتبر لوحة الإدارة (Admin Dashboard) النظام الخلفي لإدارة عمليات المطعم. مبنية بالكامل باستخدام مكونات React Client Components وتتواصل مع Server Actions لإنجاز العمليات وتحديث الواجهة مباشرة.

2. الصفحات والوظائف الرئيسية

[admin/](#) (لوحة القياس الرئيسية - Dashboard)

- **الغرض:** تقديم نظرة عامة سريعة على أداء اليوم.
- **الوظائف:**
 - عرض إجمالي مبيعات اليوم، وعدد الطلبات، والعملاء النشطين.
 - الرسوم البيانية لأداء المبيعات.
 - إشعارات بالطلبات المعلقة التي تتطلب اتخاذ إجراء.

[admin/orders/](#) (إدارة الطلبات)

- **الغرض:** معالجة الطلبات الواردة وتحديث حالاتها.
- **الوظائف:**
 - عرض جدول تفاعلي للطلبات مصنف حسب الحالة (معلق، قيد التحضير، في الطريق، ...).
 - تحديث حالة الطلب بضغط زر.
 - عرض تفاصيل الفاتورة ومسار التوصيل.
 - **ملاحظة:** هذه هي الصفحة الوحيدة المتاحة لحسابات [order_manager](#).

[admin/menu/](#) و [admin/categories/](#) (إدارة القائمة)

- **الغرض:** التحكم بالمنتجات المعروضة في الموقع والتطبيق.
- **الوظائف:**
 - إضافة، وتعديل، وحذف الأصناف (Soft Delete للطلبات المرتبطة).
 - إدارة التصنيفات وتحديد الأيقونات والصور.
 - تحديد الأحجام المتوفرة وتسعير كل حجم بشكل مستقل ([item_prices](#)).

admin/offers/ (إدارة العروض)

- **الغرض:** إنشاء حزم تسويقية وتخفيضات.
- **الوظائف:**
 - إنشاء عروض خصم نسبة، خصم مبلغ ثابت، وسعر للحزمة.
 - تحديد الأصناف المشمولة في العرض والكميات.
 - تحديد فترة صلاحية العرض (بداية ونهاية).

admin/reviews/ (إدارة التقييمات)

- **الغرض:** مراقبة جودة الخدمة.
- **الوظائف:**
 - عرض التقييمات الواردة من العملاء.
 - قبول/إخفاء التقييمات.
 - تحديد التقييمات المميزة لتظهر في الصفحة الرئيسية.

admin/delivery/ (نظام التوصيل)

- **الغرض:** التحكم المالي بآلية حساب التوصيل.
- **الوظائف:**
 - تحديد الرسوم الأساسية والمسافة المجانية ومقابل الكيلومتر الإضافي.
 - تفعيل رسوم الذروة ورسوم سوء الأحوال الجوية.
 - تحديد النطاق الأقصى المسموح لخدمة التوصيل.

admin/gallery/ (معرض الصور)

- **الغرض:** إدارة الهوية البصرية.
- **الوظائف:** رفع الصور لتظهر للعملاء في قسم المعرض المخصص.

admin/settings/ و admin/reports/

- سيتم التطرق لهما بالتفصيل في تقارير منفصلة.

3. تدفق تحديث حالة الطلب

```
sequenceDiagram
    participant Admin as Order Manager
    participant UI as /admin/orders
    participant API as updateOrderStatus Action
    participant DB as orders Table
    participant History as order_status_history

    Admin->>UI: Click "Accept Order"
    UI->>API: updateOrderStatus(orderId, 'confirmed')
    API->>DB: Check Version (Optimistic Locking)
    DB-->>API: Version Matches
    API->>DB: Update Status & Increment Version
    API->>History: Insert New Status Record
    API-->>UI: Update Successful
    UI->>Admin: Show Success Toast
```

4. الحماية والتأمين

- كل المكونات محاطة بـ **Layout** يمنع غير المصرح لهم من الرؤية (Client-Side Protection).
- مدعومة بـ **Middleware** يمنع الوصول للصفحات والـ APIs (Edge Protection).
- محصنة على مستوى **RLS** يمنع استرجاع البيانات بدون Role صحيح (Database Protection).

توثيق نظام التقارير – مشروع بيت المندي

1. نظرة عامة

تم بناء نظام التقارير كطبقة تحليلية متقدمة تهدف إلى تقديم رؤى تجارية دقيقة لاتخاذ القرارات. يعتمد النظام على معالجة البيانات من خادم قاعدة البيانات وتنسيقها للواجهة الأمامية باستخدام Rcharts للتصور البصري.

2. معمارية التقارير (Reports Architecture)

تم فصل التقارير إلى APIs مستقلة لضمان سرعة الاستجابة ومنع اختناق الخادم (Server Bottlenecks).

مسارات التقارير (API Routes):

- [api/reports/dashboard/](#): ملخص الأداء اليومي.
- [api/reports/sales/](#): تقارير المبيعات مع فلترة حسب الفترة.
- [api/reports/orders/](#): تحليلات حالات الطلب والفترات الزمنية.
- [api/reports/products/](#): إحصائيات المنتجات الأكثر والأقل مبيعاً.
- [api/reports/customers/](#): بيانات العملاء الدائمين وحجم إنفاقهم.
- [api/reports/delivery-analytics/](#): تحليل مسافات التوصيل ورسومه.

3. تدفق البيانات للتقارير (Data Flow)

sequenceDiagram

participant Admin as Manager Dashboard

participant Component as Reports UI

participant API as /api/reports/*

participant DB as Supabase PostgreSQL

Admin->>Component: Select Date Range (e.g. Last 7 Days)

Component->>API: Fetch Data (startDate, endDate)

API->>DB: Execute Aggregation Query (SUM, COUNT, GROUP BY)

DB-->>API: Raw Data Records

API->>API: Transform Data (Format Currency, Dates)

API-->>Component: JSON Array

Component->>Admin: Render Recharts Graphs & Data Tables

4. أدوات التصدير والطباعة

يتيح النظام للإدارة إمكانية تصدير البيانات للاستخدام خارج المنصة:

1. تصدير إلى Excel (**lib/exportExcel.ts**):

- يستخدم مكتبة **exceljs** لبناء جداول مخصصة.
- يدعم تصدير تقارير المبيعات، الطلبات، المنتجات.
- ينسق الأعمدة بشكل متوافق مع اللغة العربية (RTL).

2. طباعة الفواتير والتقارير (**lib/printReport.ts**):

- يستخدم مزيجاً من **html2canvas** و **jsPDF** لتوليد ملفات PDF دقيقة.
- يستخدم لطباعة فواتير الطلبات بشكل احترافي يحتوي على شعار المطعم، كود QR للتتبع، وتفاصيل الحساب.

5. ميزات الأداء (Performance Handling)

بسبب كثافة العمليات الحسابية في التقارير (Aggregation):

- يتم تنفيذ جميع حسابات التجميع (SUM, COUNT) على مستوى قاعدة البيانات PostgreSQL لتقليل حجم الذاكرة المستهلك في خادم Next.js.
- يتم تجاهل الطلبات المحذوفة (**is_deleted=true**) والمرفوضة في حسابات الأرباح لمنع تشويه البيانات.

توثيق تقييم الجودة والأداء (Quality & Performance)

1. تقييم الجودة البرمجية (Code Quality Score)

درجة الجودة الحالية: 88/100

أسباب التقييم:

نقاط القوة (95+):

- استخدام قوي لـ **TypeScript**: النظام مُعرّف الأنواع بشكل ممتاز، مما قلل من أخطاء وقت التشغيل (Runtime Errors) إلى الحد الأدنى.
- فصل الاهتمامات (Separation of Concerns): الفصل التام بين مكونات الواجهة (Components) والمنطق المالي (Pricing Engine) وعمليات قاعدة البيانات (Repositories).
- أمان المعاملات (Atomic Transactions): استخدام **db.transaction()** في عملية إنشاء الطلب يحمي قاعدة البيانات من التناقض.
- تجنب التبعيات الثقيلة (Bundle Optimization): استخدام **Native Fetch** API لتوجيه، وتجنب استخدام مكتبات ضخمة بلا داعٍ.

خصومات التحسين المتبقية (7-):

- بعض المكونات في **HomeClient.tsx** كبيرة جداً وكان يُفضل تقسيمها إلى مكونات أصغر.
- بعض الإعدادات مخزنة كقوائم **jsonb** في قاعدة البيانات مما يعقد قليلاً من الاستعلام المباشر ويحتاج لمعالجة برمجية.

2. تحليل الأداء (Performance Analysis)

2.1 Server Side Performance

- خوادم **Vercel** Serverless تتعامل بكفاءة مع **Server Actions**.
- استخدام **supabase/ssr** قلل زمن انتقال الجلسات بين صفحات (Server Components).
- سرعة الاستعلامات: ممتازة، بفضل استخدام **Indexes** في تصميم **Drizzle** للمفاتيح الأجنبية وأرقام الهواتف (phone).

Client Side Performance 2.2

- **Bundle Size:** حجم ملفات جافاسكريبت المحملة جيد. استخدام **Tailwind CSS** منع تحميل أي CSS غير مستخدم.
- **Dynamic Imports:** النظام لا يستخدم حالياً الاستيراد الديناميكي (**next/dynamic**) بكثافة، مما يمثل فرصة للتحسين لتقليل وقت التفاعل الأولي (TTI)، خاصة لمكتبات المخططات مثل **recharts**.

Caching Strategy 2.3

- يعتمد المشروع على التخزين المؤقت التلقائي المدمج في **fetch** الخاص بـ Next.js.
- صفحات القائمة (**/menu**) والصور يتم معالجتها بـ Next/Image للحصول على ضغط **WebP** و **Lazy Loading** تلقائي.

3. تحليل هيكلية الملفات (Project Structure)

الوصف الوظيفي للمجلدات الأساسية:

- **/src/app/**: الهيكل التوجيهي، كل مجلد هنا يمثل مساراً (Route).
- **/src/actions/**: قلب العمليات (التي تتطلب حماية الخادم وتعديل البيانات).
- **/src/components/**: مكعبات البناء (UI Blocks) التي لا تهتم بتوجيه المسارات.
- **/src/db/**: توصيف البيانات (Schema)، الاتصال بقاعدة البيانات، وتطبيق السياسات الأمنية.
- **/src/lib/**: الدماغ المفكر (العقول المستقلة)، أي كود حسابي أو خوارزمية (مثل التوجيه والأسعار) يجب أن تكون هنا.
- **/src/repositories/**: طبقة عزل بين الـ Server Actions و Drizzle لتسهيل تبديل قواعد البيانات لو لزم الأمر.

4. فرص التوسع والتحسين (Scalability)

النظام الحالي جاهز لدعم الفروع الإضافية برمجياً.

- لتحقيق التوسع المستقبلي: يُفضل تحويل لوحة التقارير المعقدة إلى Materialized Views داخل PostgreSQL لضمان سرعة الاستعلامات في حالة نمو قاعدة البيانات لتشمل مئات الآلاف من الطلبات.

تقييم SEO و PWA – مشروع بيت المني

1. تحليل تحسين محركات البحث (SEO Assessment)

تم تجهيز المنصة بأفضل ممارسات الـ SEO الحديثة الخاصة بـ 14 Next.js لضمان أعلى ظهور في محركات البحث.

:Open Graph و Metadata

- ملف **layout.tsx**: يحتوي على كائن **metadata** شامل.
- يدعم تخصيص العنوان ديناميكياً لكل صفحة مع قالب (**s% | مطعم بيت المني**).
- يحتوي على الكلمات المفتاحية المتعلقة بالمطاعم والمأكولات اليمينية.
- إعدادات **Open Graph** و **Twitter Cards** لتوليد معاينات جذابة عند مشاركة الروابط في منصات التواصل (واتساب، فيسبوك، تويتر).

البيانات المنظمة (JSON-LD - Structured Data):

- تم حقن **restaurantSchema** من النوع **Restaurant** و **LocalBusiness** مباشرة في الـ **<head>**.
- يعرّف هذا لمحرك بحث جوجل: الموقع، أوقات العمل، قائمة الطعام، الإحداثيات الجغرافية، والأسعار.
- يزيد من فرصة الظهور في خرائط جوجل وصناديق المعرفة (Knowledge Graph).

ملفات الروبوتات والخرائط:

- robots.ts**: يسمح بالزحف (**User-Agent**: * مع **Allow**: /) ويمنع العناكب من الدخول لمسار الإدارة **admin/**.
- sitemap.ts**: يولّد الخريطة ديناميكياً لتشمل المسارات الأساسية وتاريخ آخر تحديث ومعدل التغيير وأهمية الصفحة (**Priority**).

2. تحليل PWA (تطبيقات الويب التقدمية)

بناءً على الفحص، البنية الأساسية الحالية للمشروع **تفتقر** إلى الإعدادات الكاملة لـ PWA.

الميزات الموجودة حالياً:

- تصميم متجاوب (Responsive) تماماً مع شاشات الجوال.

- وجود ملف **manifest** افتراضي ضمن Next.js.
- أيقونات (Apple Touch Icons) محددة في **layout.tsx**.

الميزات المفقودة المطلوبة لـ PWA كامل:

1. **Service Worker**: لا يوجد Service Worker مسجل لتفعيل ميزات الـ Offline أو التخزين المؤقت المتقدم.
 2. ملف **manifest.json** مستقل ومفصل: يفضل أن يحتوي على ألوان الشيم وتحديد الـ **display: standalone** بدقة ليعمل كتطبيق جوال.
 3. تثبيت التطبيق (Add to Home Screen): لا توجد شاشة أو موجه يطالب المستخدم بتثبيت التطبيق.
- الخلاصة للـ PWA:** النظام حالياً يعطي تجربة "موقع موبايل" ممتازة ولكنه ليس PWA بالمعنى التقني الكامل. يُنصح بإضافة حزمة **next-pwa** لتفعيل هذه الخصائص.

التقرير المعماري النهائي للمشروع (Final)

(Architecture Report)

1. الهيكلية الحالية ونقاط القوة

تمتاز منصة بيت المندي بتصميم حديث وموثوق (Robust Design):

- فصل المنطق (Decoupling):** عزل مهام الأسعار وحسابات التوصيل كلياً عن مكونات الواجهة (UI)، مما رفع من مستوى الأمان المالي.
- أداء مرتفع (High Performance):** دمج الخادم والعميل من خلال Server Actions يتيح سرعة هائلة في المعاملات دون الحاجة لإدارة حالة API مفرطة (No redundant state sync).
- أمان متطور (Advanced Security):** استخدام RLS يمنع فعلياً أي هجوم لاستخراج البيانات (Data Exfiltration) أو التلاعب بها (Tampering)، حتى لو تم كشف نقاط الاتصال (Endpoints).

2. فرص التحسين المستقبلية (Opportunities)

بالرغم من القوة المعمارية، توجد بعض المجالات القابلة للتطوير في النسخ القادمة:

- PWA Integration:** إضافة Service Worker متكامل لتمكين المستخدمين من فتح القائمة بدون اتصال إنترنت، وتثبيت المنصة كتطبيق حقيقي على هواتفهم.
- Caching المتقدم:** توظيف Redis لحفظ الجلسات النشطة وعربات التسوق المؤقتة لتخفيف الضغط المتكرر عن PostgreSQL.
- نظام الإشعارات:** استخدام WebSockets (وهو متاح عبر Supabase Realtime) بشكل أعمق لإرسال تنبيهات (Push Notifications) للإدارة وللعملاء عند تغير الحالات.
- تجزئة الكود (Code Splitting):** استخدام next/dynamic لتحميل المكونات الثقيلة (مثل المخططات البيانية Recharts وخرائط Leaflet) فقط عندما يراها المستخدم (Lazy Loading).

3. التوثيق المرجعي

- تم إنتاج هذا المستند لتحليل بيئة الإنتاج الخاصة بمطعم بيت المندي.
- أثبت النظام جاهزيته للعمل والتحمل (Production-Ready) مع تغطية شاملة لمعالجة الأخطاء والعمليات الحساسة.

- يمكن الرجوع إلى الوثائق السابقة (الأمان، قاعدة البيانات، التوصيل، الإدارة، الطلبات، والتقارير) لفهم التفاصيل الدقيقة لكل وحدة.

نهاية التوثيق التقني - إنتاج Esnaad Tech

المراجع التقنية

- توثيق Next.js الرسمي: <https://nextjs.org/docs>
- Supabase: <https://supabase.com/docs>
- Drizzle ORM: <https://orm.drizzle.team/docs/overview>
- خوارزمية Haversine: https://en.wikipedia.org/wiki/Haversine_formula
- خدمة OSRM للتوجيه: <http://project-osrm.org>